

对状态进行格雷码赋值。取三位编码 XYZ , 对状态1、0、00、000、0000 + 分别赋值为 000, 001, 011, 010, 100。状态表如下:

Table 1: 状态转移表

当前状态 XYZ	下一个状态		输出	
	$I = 0$	$I = 1$	$I = 0$	$I = 1$
000	001	000	0	0
001	011	000	0	0
011	010	000	0	0
010	100	000	0	0
100	100	000	1	0

注意到存在 3 个无效状态编码：101、110、111，这些无效状态编码与输入可形成 6 种无效的组合，我们将其当成无关最小项来处理。

后面计划选择 D 触发器实现该电路。列出该电路的状态转移方程（触发器的输入方程）和输出方程。触发器的输入方程和输出方程可以表示为当前状态变量和输入变量的最小项之和：

$$X(t+1) = D_X(X, Y, Z, I) = \Sigma m(4, 8)$$

$$Y(t+1) = D_Y(X, Y, Z, I) = \Sigma m(2, 6)$$

$$Z(t+1) = D_Z(X, Y, Z, I) = \Sigma m(0, 2)$$

$$O(X, Y, Z, I) = \Sigma m(8)$$

利用卡诺图进行优化，得到优化后的触发器输入方程和输出方程。

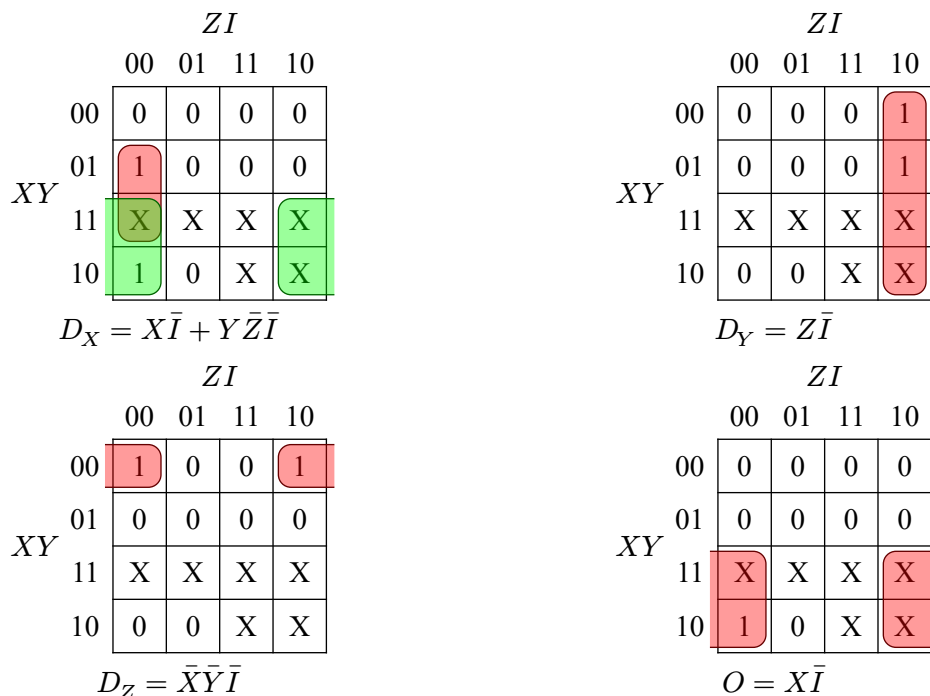


Figure 2: 具有 D 触发器的格雷码序列识别器的卡诺图

触发器的输入方程和输出方程如下：

$$D_X = X\bar{I} + Y\bar{Z}\bar{I}$$

$$D_Y = Z\bar{I}$$

$$D_Z = \bar{X}\bar{Y}\bar{I}$$

$$O = X\bar{I}$$

3. 电路实现

该部分通过工艺映射, 将逻辑表达式转化为电路图。在 Logisim 中绘制电路图, 电路图如下:

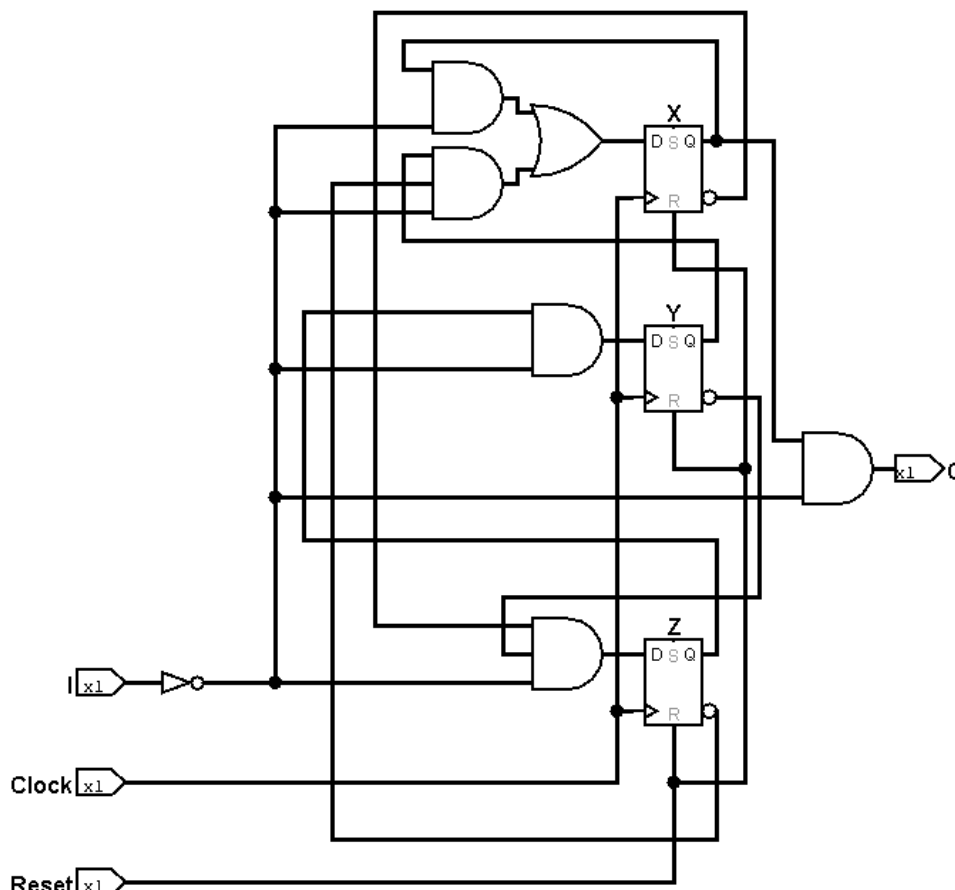


Figure 3: 具有 D 触发器的格雷码序列识别器的逻辑电路图

下面是电路的 Verilog 代码, 在行为描述中实现了该电路。代码中分离了时序逻辑 (状态的更新) 和组合逻辑 (下一状态和输出的计算)。通过使用 `parameter` 定义状态、`always @(posedge clk or posedge rst)` 实现时序更新、`always @(*)` 实现组合逻辑, 以及 `case` 和 `if/else` 语句描述逻辑, 有效地描述了电路行为。

代码中部分注释 (如前面的基本信息) 过于冗长, 因此没有放在报告当中。

```
// zero_sequence_detector.v

`timescale 1ns / 1ps

// 定义模块
module zero_sequence_detector (
```

```

input wire clk,    // 时钟信号
input wire rst,    // 复位信号, 高电平有效
input wire I,      // 输入信号
output reg  O      // 输出信号
);

// 定义状态, 使用 parameter 关键字提高可读性
parameter STATE_1      = 3'b000; // 对应没有0的状态
parameter STATE_0      = 3'b001; // 对应1个0的状态
parameter STATE_00     = 3'b011; // 对应2个0的状态
parameter STATE_000    = 3'b010; // 对应3个0的状态
parameter STATE_0000_PLUS = 3'b100; // 对应4个及以上个0的状态

// 声明状态寄存器和下一状态线
reg [2:0] current_state; // 当前状态
reg [2:0] next_state;    // 计算出的下一状态

// =====
// 时序逻辑: 状态寄存器的更新
// 在时钟上升沿或复位信号上升沿触发
// =====
always @(posedge clk or posedge rst) begin
    if (rst) begin
        // 复位时, 回到初始状态
        current_state <= STATE_1;
    end else begin
        // 非复位时, 在时钟上升沿更新当前状态为下一状态
        current_state <= next_state;
    end
end

// =====
// 组合逻辑: 计算下一状态
// 根据当前状态和输入信号, 计算下一个状态
// 使用 case 语句描述状态转移表
// =====
always @(*) begin
    // 默认下一状态保持当前状态 (可选, 但有助于处理未指定的情况)
    next_state = current_state; // 默认值, 将在 case 语句中覆盖

    case (current_state)
        STATE_1: begin
            if (I == 0) begin
                next_state = STATE_0;
            end else begin // I == 1
                next_state = STATE_1;
            end
        end
        STATE_0: begin
            if (I == 0) begin
                next_state = STATE_00;
            end else begin // I == 1
                next_state = STATE_000;
            end
        end
        STATE_00: begin
            if (I == 0) begin
                next_state = STATE_000;
            end else begin // I == 1
                next_state = STATE_0000_PLUS;
            end
        end
        STATE_000: begin
            if (I == 0) begin
                next_state = STATE_0000_PLUS;
            end else begin // I == 1
                next_state = STATE_000;
            end
        end
        STATE_0000_PLUS: begin
            if (I == 0) begin
                next_state = STATE_0000_PLUS;
            end else begin // I == 1
                next_state = STATE_000;
            end
        end
    endcase
end

```

```

        next_state = STATE_1;
    end
end

STATE_00: begin
    if (I == 0) begin
        next_state = STATE_000;
    end else begin // I == 1
        next_state = STATE_1;
    end
end

STATE_000: begin
    if (I == 0) begin
        next_state = STATE_0000_PLUS;
    end else begin // I == 1
        next_state = STATE_1;
    end
end

STATE_0000_PLUS: begin
    if (I == 0) begin
        next_state = STATE_0000_PLUS; // 保持在结束状态
    end else begin // I == 1
        next_state = STATE_1;
    end
end

// 处理无效状态编码（通常不需要显式写出，综合工具会根据需求处理，
// 但为了完整性，可以指定无效状态的转移，例如回到初始状态）
default: begin
    next_state = STATE_1; // 无效状态转移到初始状态
end
endcase
end

// =====
// 组合逻辑：计算输出
// 根据当前状态和输入信号，计算输出信号 O
// =====
always @(*) begin
    // 默认输出为 0
    O = 1'b0;

    // 根据状态和输入确定输出
    if (current_state == STATE_0000_PLUS && I == 0) begin
        O = 1'b1;
    end
    // 其他情况输出默认为 0
end

endmodule

```

4. 电路验证

a) TestBench

编写 Verilog TestBench, 对实现的电路模块进行功能验证。TestBench 中包含了时钟信号的生成、复位信号的控制、输入信号的激励以及输出信号的监控。

TestBench 设计了 5 个测试序列, 分别对应不同的输入信号组合。每个测试序列都在时钟周期内进行激励, 并在每个时钟周期后观察输出信号 O 的变化。通过观察输出信号 O 的变化, 验证电路是否按照预期工作。

```
// testbench.v

// TestBench for zero_sequence_detector

`timescale 1ns / 1ps // 定义仿真时间单位和精度

module zero_sequence_detector_tb;

    // 信号声明
    reg clk;
    reg rst;
    reg I;
    wire O;

    // 参数定义
    parameter PERIOD = 10; // 时钟周期 10ns (100MHz)

    // 待测试模块例化 (DUT)
    zero_sequence_detector dut (
        .clk(clk),
        .rst(rst),
        .I(I),
        .O(O)
    );

    // 时钟生成
    initial begin
        clk = 0;
        forever begin
            #(PERIOD / 2) clk = ~clk;
        end
    end

    // 激励生成
    initial begin
        // 打开波形文件
        $dumpfile("zero_sequence_detector.vcd");
        $dumpvars(0, zero_sequence_detector_tb);

        // 初始化输入信号
        I = 0;
```

```

// 1. 进行复位
rst = 1;
#PERIOD; // 保持复位一个时钟周期
rst = 0;
#PERIOD; // 等待一个时钟周期，确保复位完成

// 2. 模拟不同的输入序列

$display("--- Starting Simulation ---");
// 监控关键信号
$monitor("Time=%0t, State=%b, rst=%b, I=%b, O=%b", $time, dut.current_state, rst,
I, 0);

// 测试序列1: 0000 (应输出1)
#PERIOD; I = 0; // 第一个0, 状态应转到 001
#PERIOD; I = 0; // 第二个0, 状态应转到 011
#PERIOD; I = 0; // 第三个0, 状态应转到 010
#PERIOD; I = 0; // 第四个0, 状态应转到 100, 输出应为1

// 测试序列2: 1 (应复位状态到 000)
#PERIOD; I = 1;

// 测试序列3: 0010000 (测试复位和再次检测)
#PERIOD; I = 0; // 001
#PERIOD; I = 0; // 011
#PERIOD; I = 1; // 1, 复位到 000
#PERIOD; I = 0; // 001
#PERIOD; I = 0; // 011
#PERIOD; I = 0; // 010
#PERIOD; I = 0; // 100, 输出应为1

// 测试序列4: 保持在 0000+ 状态输入 0
#PERIOD; I = 0; // 保持在 100, 输出应为1

// 测试序列5: 从 0000+ 状态输入 1
#PERIOD; I = 1; // 复位到 000

// 3. 结束仿真
# (PERIOD * 5); // 再运行一段时间
$display("--- Ending Simulation ---");
$finish; // 结束仿真

end

endmodule

```

b) 仿真结果

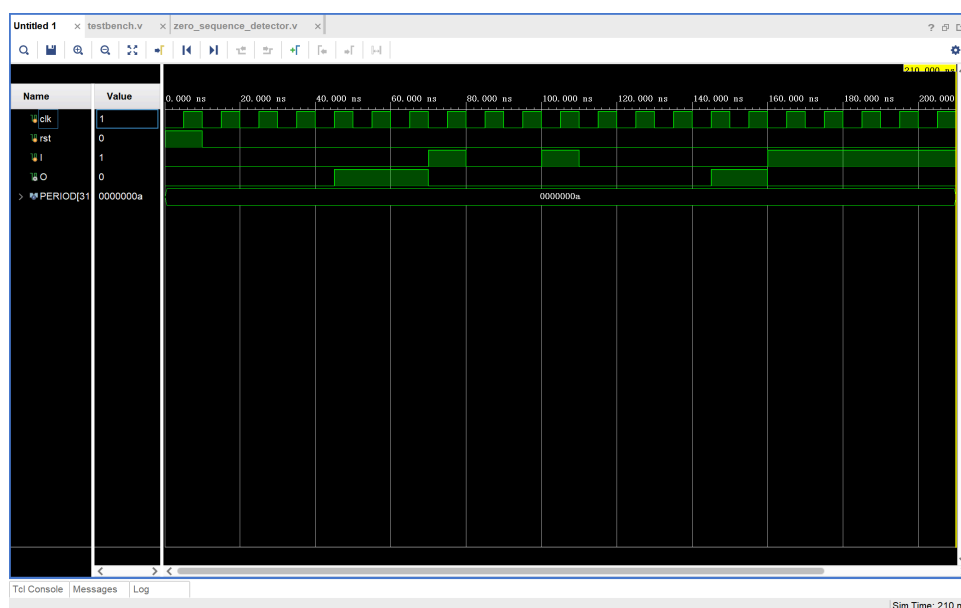


Figure 4: 仿真波形图

仿真结果中显示了 5 个测试序列的输入信号 I 和输出信号 O 的变化。可以看到, 当输入信号 I 连续为 0 时, 输出信号 O 为 1; 当输入信号 I 为 1 时, 输出信号 O 为 0。仿真结果符合预期, 验证了电路的正确性。

5. 实验心得

本次实验通过设计和实现一个串行数据子序列检测器, 进一步加深了我对时序电路设计流程的理解。从状态分析、状态分配、状态转移图绘制, 到卡诺图化简和 Verilog 实现, 每一步都锻炼了我的逻辑思维和工程实践能力。进一步的, 通过 Logisim 绘制电路图、编写 Verilog 代码和 Verilog TestBench, 并在 Vivado 中进行仿真验证, 让我体会到理论与实际结合的重要性。

实验过程中遇到了一些状态转移和输出逻辑的细节问题, 通过查阅资料和调试逐步解决, 提升了独立分析和解决问题的能力。整体而言, 本次实验不仅巩固了数字逻辑基础知识, 也为后续更复杂的时序电路设计打下了坚实基础。